RTL INNOVATE: Digital VLSI Hackfest

organised by EVDC Club in collaboration with
Department of ECE and ECE VSI D&T

Project Title

Design of a Vending Machine Controller with multiple products and dynamic pricing

Team CREATIONISTS

Members:

Abhishek Tyagi (ECE 2nd Year)

Dilip Yadav (ECE 2nd Year)

Introduction to the Problem Statement:

A Vending Machine is required, which can provide 3 or more products at the same time keeping in mind that the user will input any amount of coins that can be less than, greater than or equal to the price of the product.

User should get the change as remaining balance after purchase.

Objective:

Min. no. of products offered = 3

Functional Behaviour --

Money handling:

- User inserts coins → Balance increases.
- System tracks total balance.

Product Selection:

- If balance >= price → dispense the product
- Deduct price from the balance
- Return remaining change

Dynamic Pricing:

- Input-controlled prices (iterable)

Invalid Conditions:

- Insufficient balance → no dispense
- Invalid selection → ignore the input

Verilog Code:

```
module creationists(
   input clk, rst,
   input [1:0] select,
   input [7:0] coin, prodA, prodB, prodC,
   input insert_coin,
   output reg [7:0] change,
   output reg dispense
);

   reg [1:0] state; //2 bit variable for state of ven. machine
   reg [7:0] balance,price;//8 bit input for balance and price of the
//product we have selected


   parameter IDLE     = 2'd0;
   parameter CHECK    = 2'd1;
   parameter S_DISPENSE = 2'd2;//parameters for different states of
//machine


   always @(*) begin
      case(select)
         2'd0:    price = prodA;
         2'd1:    price = prodB;
         2'd2:    price = prodC;
         default: price = 8'd0;
      endcase
   end
```

```
always @(posedge clk or posedge rst) begin
   if(rst) begin
      state    <= IDLE;
      balance  <= 8'd0;
      dispense <= 1'b0;
      change   <= 8'd0;
   end
   else begin
      case(state)


         IDLE: begin //idle state
            dispense <= 1'b0;
            if(insert_coin)
               balance <= balance + coin;
            else if (balance > 0)
               state <= CHECK;
         end

         CHECK: begin  //condition for check state
            if(balance >= price)
               state <= S_DISPENSE;
            else begin
               balance <= 8'd0;
               state   <= IDLE;
            end
         end

         S_DISPENSE: begin  //condition for dispense state
            dispense <= 1'b1;
            change   <= balance - price;
            balance  <= 8'd0;
            state    <= IDLE;
         end
```

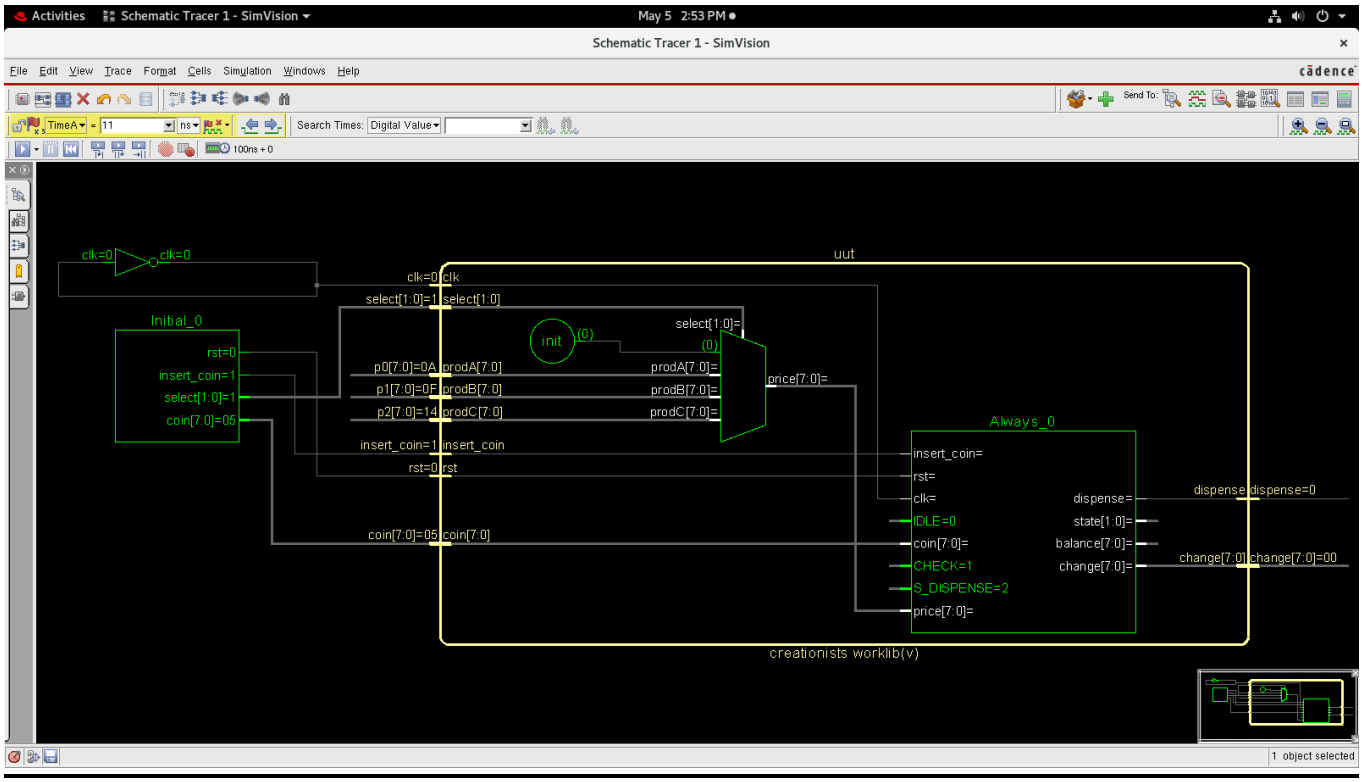
```
            default: state <= IDLE;
          endcase
        end
    end
endmodule
```

Testbench:

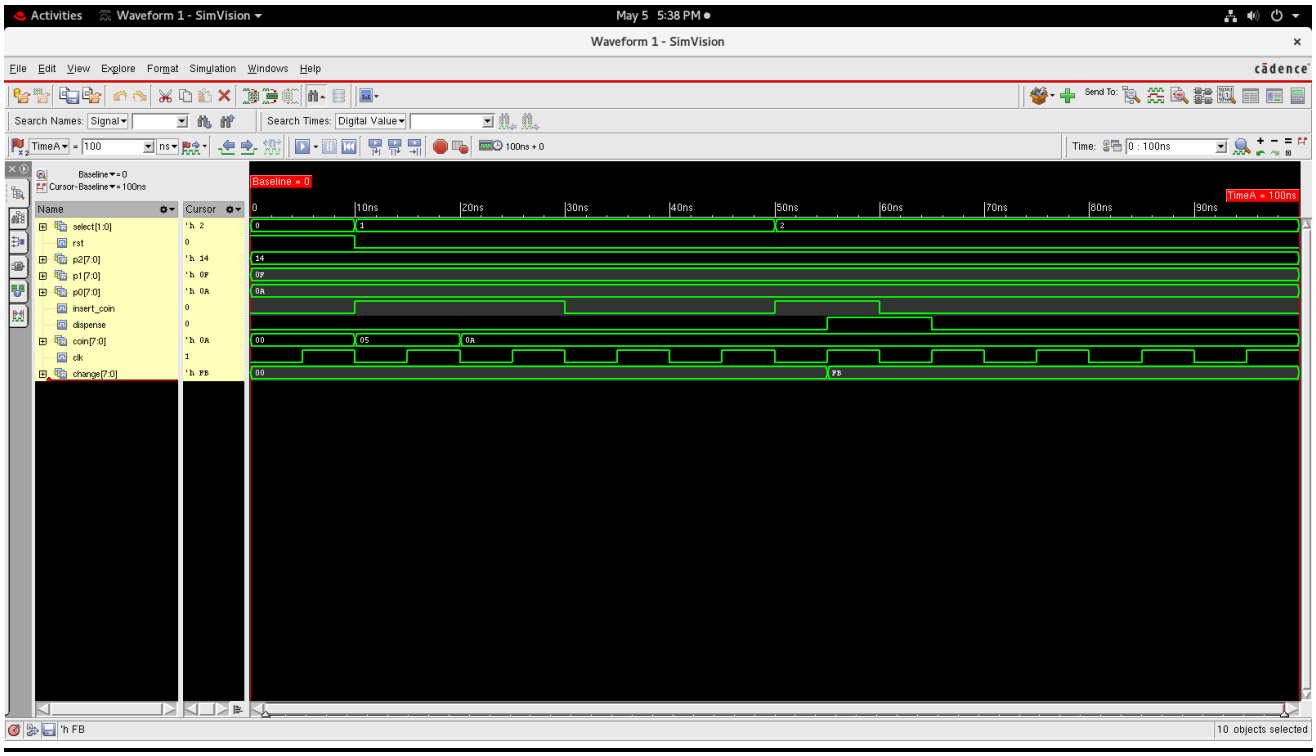
```
//`timescale 1ns/1ps
 module tb_creationists;

    reg clk=0, rst=0;
    reg [1:0] select=0;
    reg [7:0] coin=0;
    reg insert_coin=0;
    reg [7:0] p0=10, p1=15, p2=20;
    wire dispense;
    wire [7:0] change;

     creationists uut(.clk(clk), .rst(rst), .select(select), .coin(coin),
.insert_coin(insert_coin), .prodA(p0), .prodB(p1), .prodC(p2),
.dispense(dispense), .change(change));

    always #5 clk = ~clk;


    initial begin
        rst = 1; #10 rst = 0;
        select = 2'd1;
        coin=5; insert_coin=1; #10 insert_coin=0;
        coin=10; insert_coin=1; #10 insert_coin=0;
        #20;

        select = 2'd2;
```

```
        coin=10; insert_coin=1; #10 insert_coin=0;

        #40 $finish;
    end
initial begin
        $display("\nTime\tSelect\tIns\tCoin\t| Disp\tChange");

        $monitor("%0t\t%d\t%b\t%d\t| %b\t%d",
                $time, select, insert_coin, coin, dispense, change);
    end

endmodule
```

Block Diagram:



Waveform:



Challenges faced:

- FSM Complexity
- Code complexity due to Multiple input-controlled variables

Github Repository Link: Click to see the Github Repo